

Advanced Natural Language Processing: AdvNLP

*Lesson 13: Representation and processing of
speech with neural networks*

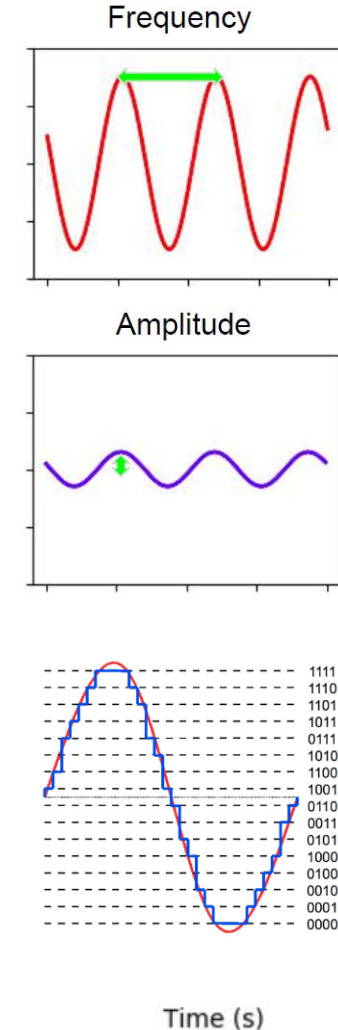
Table of contents

1. Audio and speech data
2. Architectures of speech recognition systems
3. Self-supervised learning in the wav2vec 1.0 and 2.0 encoder
4. CTC loss and CTC decoding
5. Encoder-decoder model: Whisper
6. Evaluation and current performance levels

1. AUDIO AND SPEECH DATA – AUTOMATED SPEECH RECOGNITION (ASR)

Audio & speech data

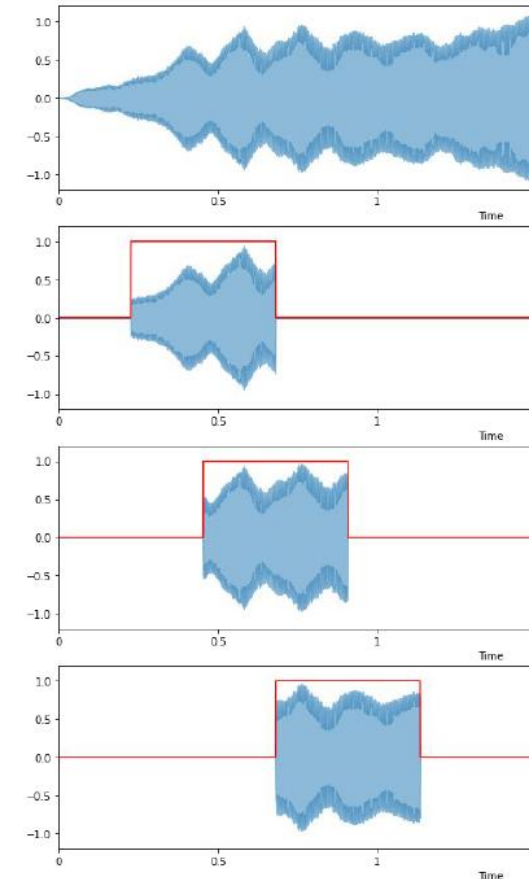
- Perceived frequency or pitch varies as the logarithm of the actual frequency (in Hz); our pitch perception is logarithmic
- Perceived amplitude or loudness varies as the logarithm of the real intensity (in dB); $dB(I) = 10 \cdot \log_{10}(\frac{I}{I_{min}})$ with $I_{min} = 10^{-12} W/m^2$
- Analog-digital conversion samples signals with frequency $1/T$ and discretely represents amplitudes with a given bit depth
 - 4-bit coding (16 possible levels), 16-bit coding (65'536 possible levels)
 - Nyquist limit = half the sampling rate is the highest frequency that can be captured from the signal
 - sampling process introduces noise, the higher the bit depth, the smaller this quantization noise



Source: <https://github.com/musikalkemist/AudioSignalProcessingForML>

Audio & speech data

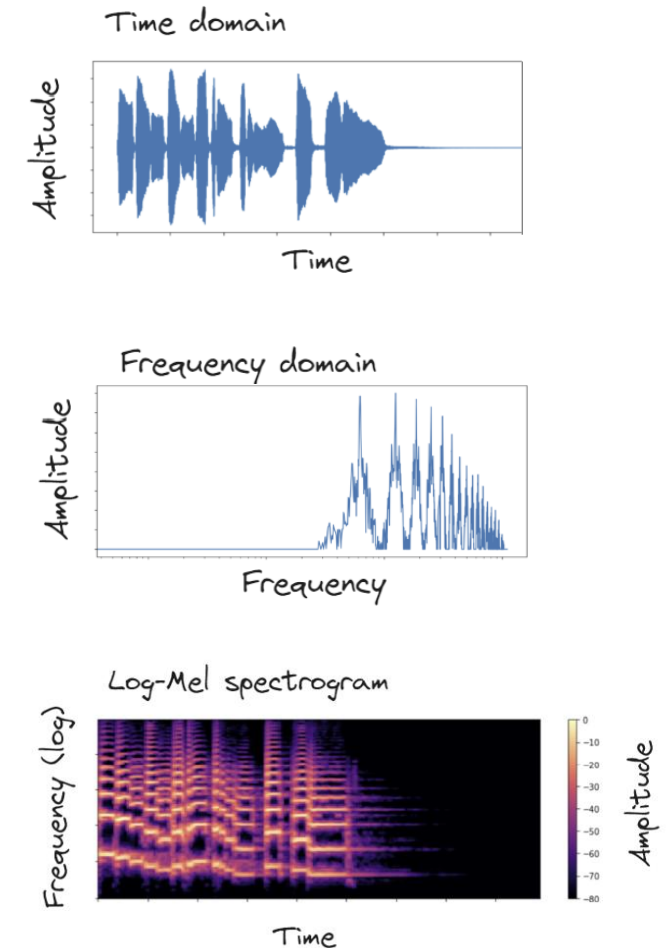
- Many audio signals are periodical and stable when considered over a very short **time window**
- Audio extracted from a time window is called a **frame** and **longer audios** can be split into consecutive, partially overlapping frames
- **Audio decomposition** is characterized by the frame's **size** (width of the red rectangles), **stride** (distance in x-direction between starting points of red rectangles) and **shape** (e.g., rectangular window)



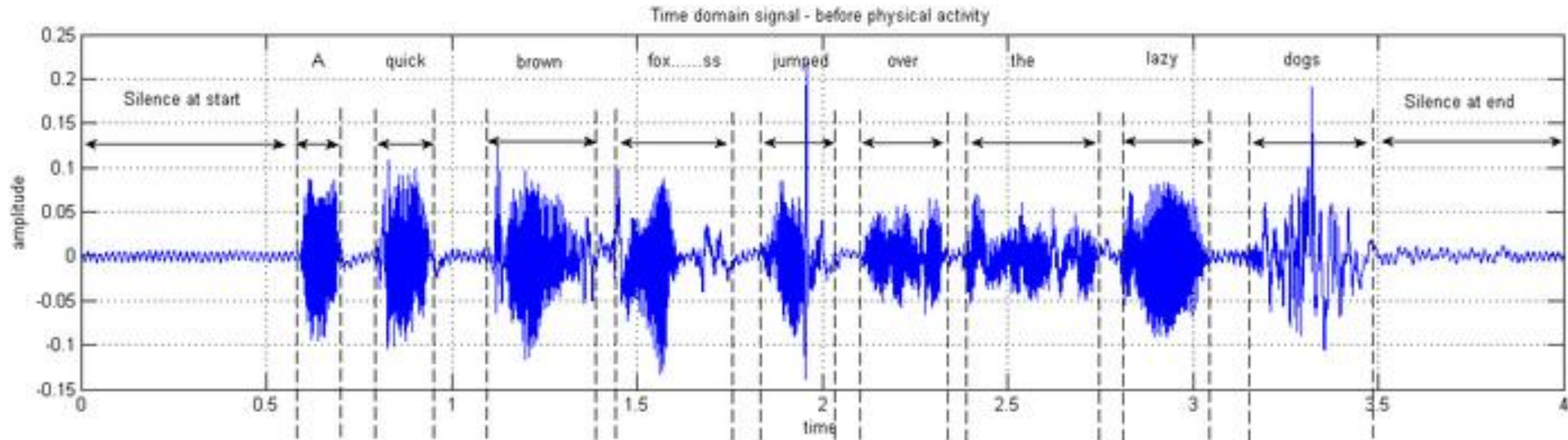
Source: <https://github.com/musikalkemist/AudioSignalProcessingForML>

Audio & speech data

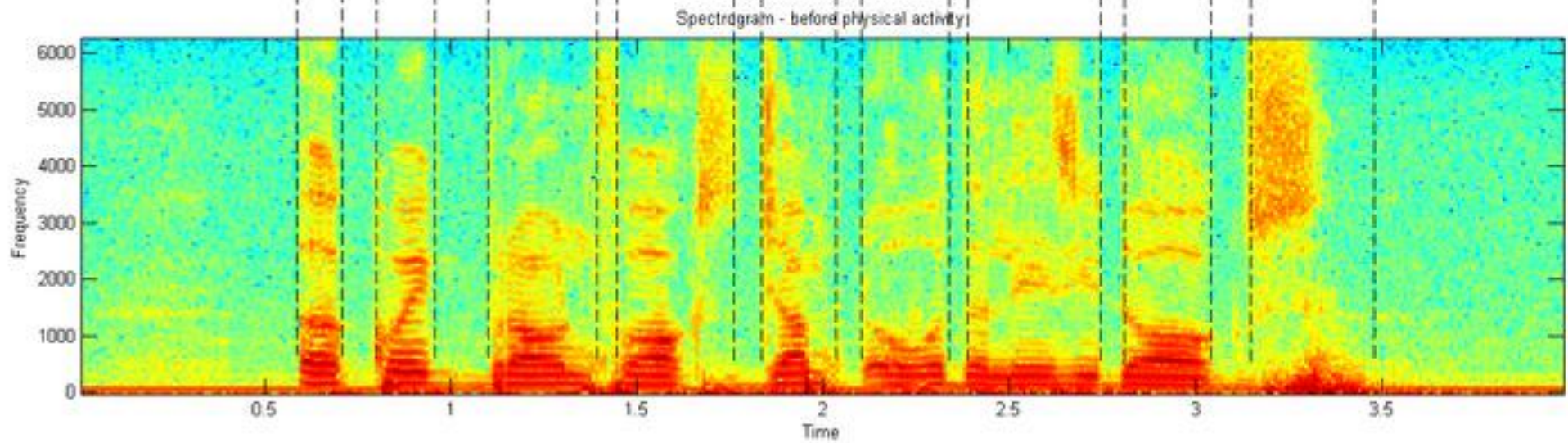
- Amplitude changes of audios are visualized in **waveform** in the **time domain**
- Audios can be decomposed within a **fixed time window** and visualized as **frequency spectrum** describing individual **frequencies** and **strengths** using **Fourier Analysis (FFT)**
- Audio **frequency changes across time** can be combined into a **spectrogram** using **multiple time windows**
- **Log-Mel spectrograms** use the “mel” scale to approximate the non-linear frequency response of the human ear
 - mel bins define a set of frequency ranges that divide the spectrum into perceptually meaningful components, using a set of filters with specific shape and spacing



Signal

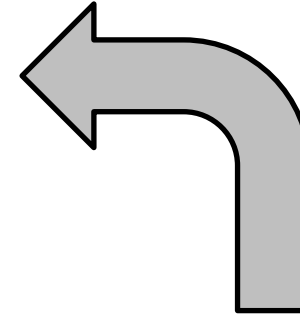
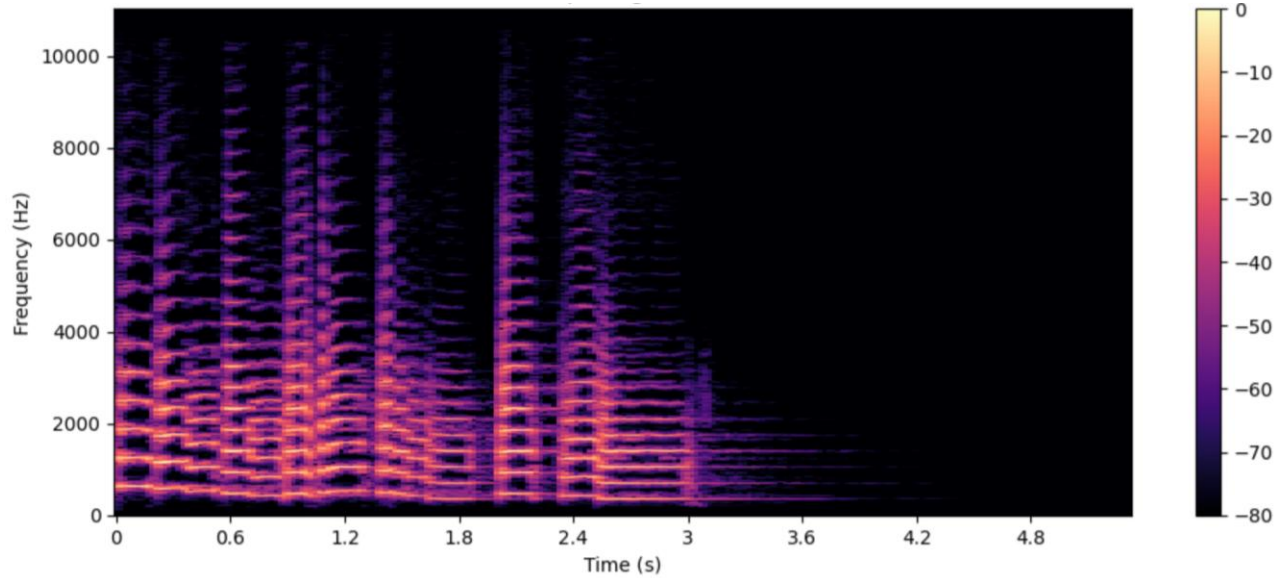


Spectrogram



Color = intensity (dB)

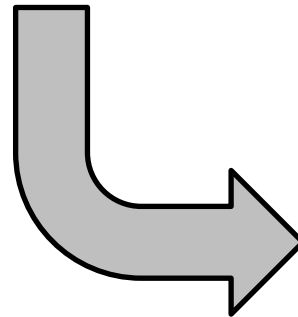
Spectrogram



$$f = 700 \left(10^{m/2595} - 1 \right)$$

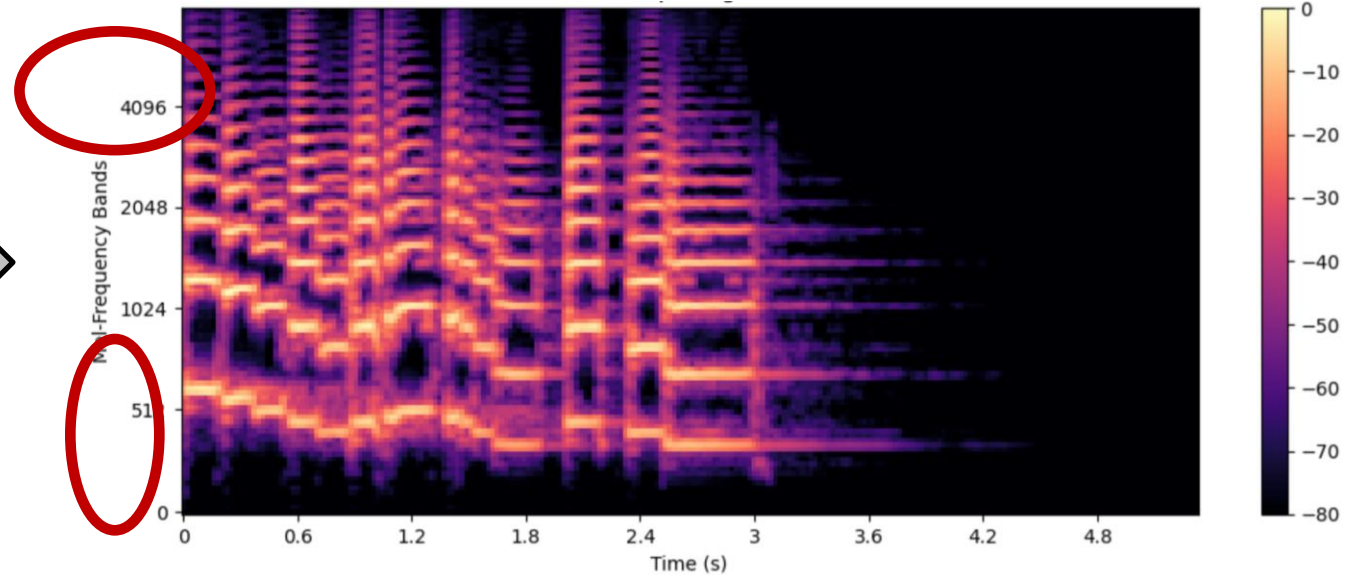
Mel Spectrogram

$$m = 2595 \log_{10} \left(1 + \frac{f}{700} \right)$$



High frequencies compressed
(i.e. lower resolution)

Low frequencies expanded
(i.e. higher resolution)



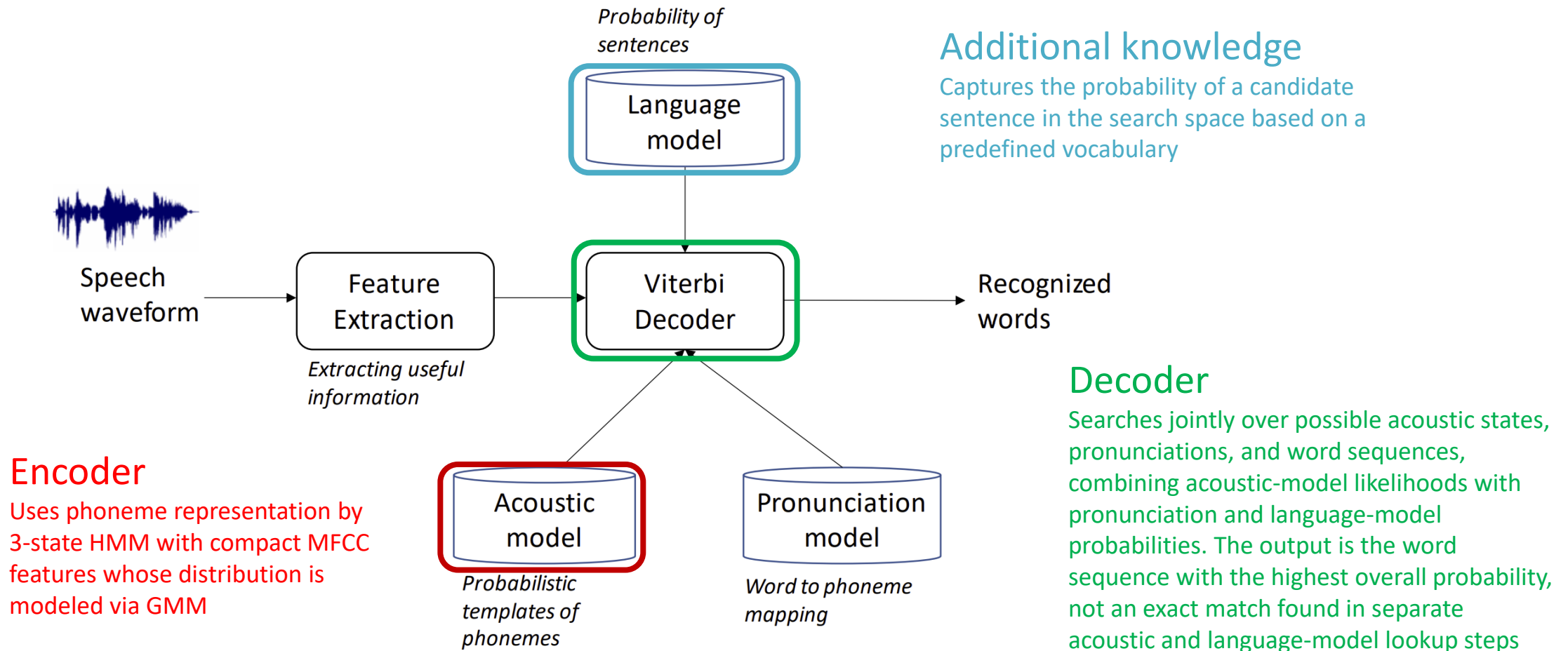
2. ARCHITECTURES OF SPEECH RECOGNITION SYSTEMS

Automated Speech Recognition (ASR)

- ASR refers to the computational process of converting **audio signal of** spoken language into **written text** (often referred to as speech-to-text (STT))
- **Similarly** to neural machine translation of text, ASR
 - has evolved: rule-based → statistical models → neural networks
 - can be modeled as **sequence-to-sequence** (seq2seq) tasks with an **encoder** using local context to build low-level representation of speech and a **decoder** using low-level representation and a larger context to find the most likely letters/ words and needs a decoding procedure
 - requires large-scale, domain-relevant training data
- **Differently** to neural machine translation of text, ASR
 - has much longer input sequences (e.g., 2 s → 200 frames (10 ms each) $\approx$ 10 letters $\approx$ 1 word)
 - poses challenges of input-output alignment, i.e., mapping of audio frames to letters/words

Automated Speech Recognition (ASR)

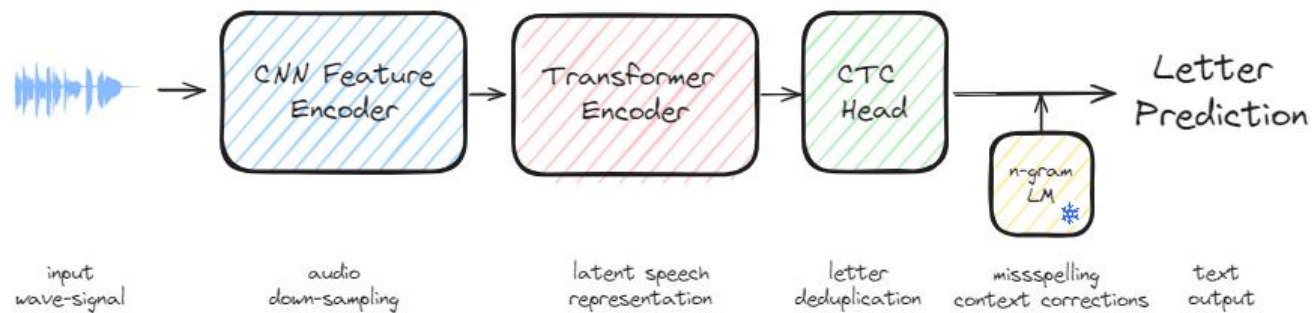
Statistical modeling for ASR



Automated Speech Recognition (ASR)

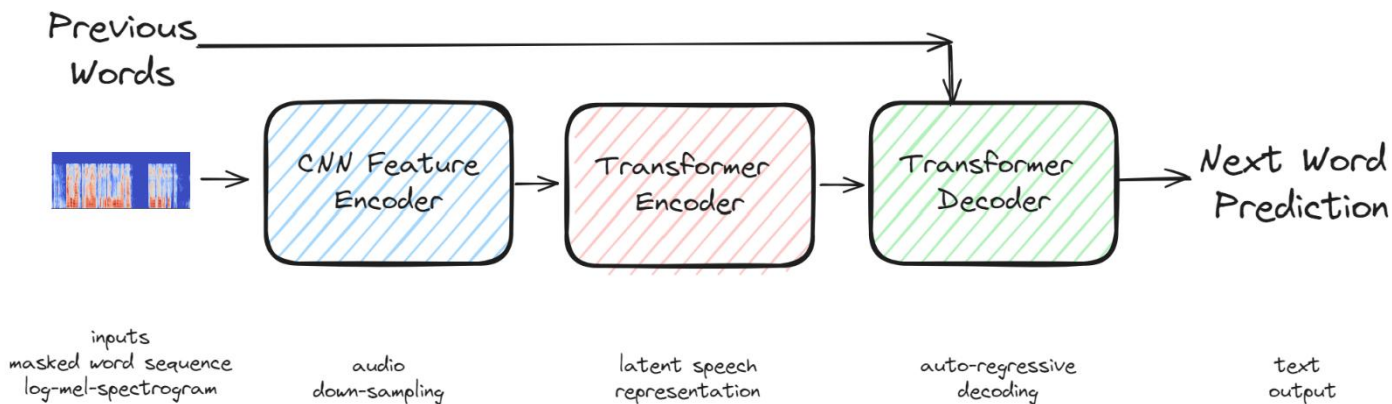
- Modern ASR models use architectures with “**encoder**” & “**decoder**” components, which share Transformer components used in neural machine text translation

wav2vec 2.0



Processes wave signal via low-level feature encoder and transformer encoder; does not use auto-regressive decoder

Whisper

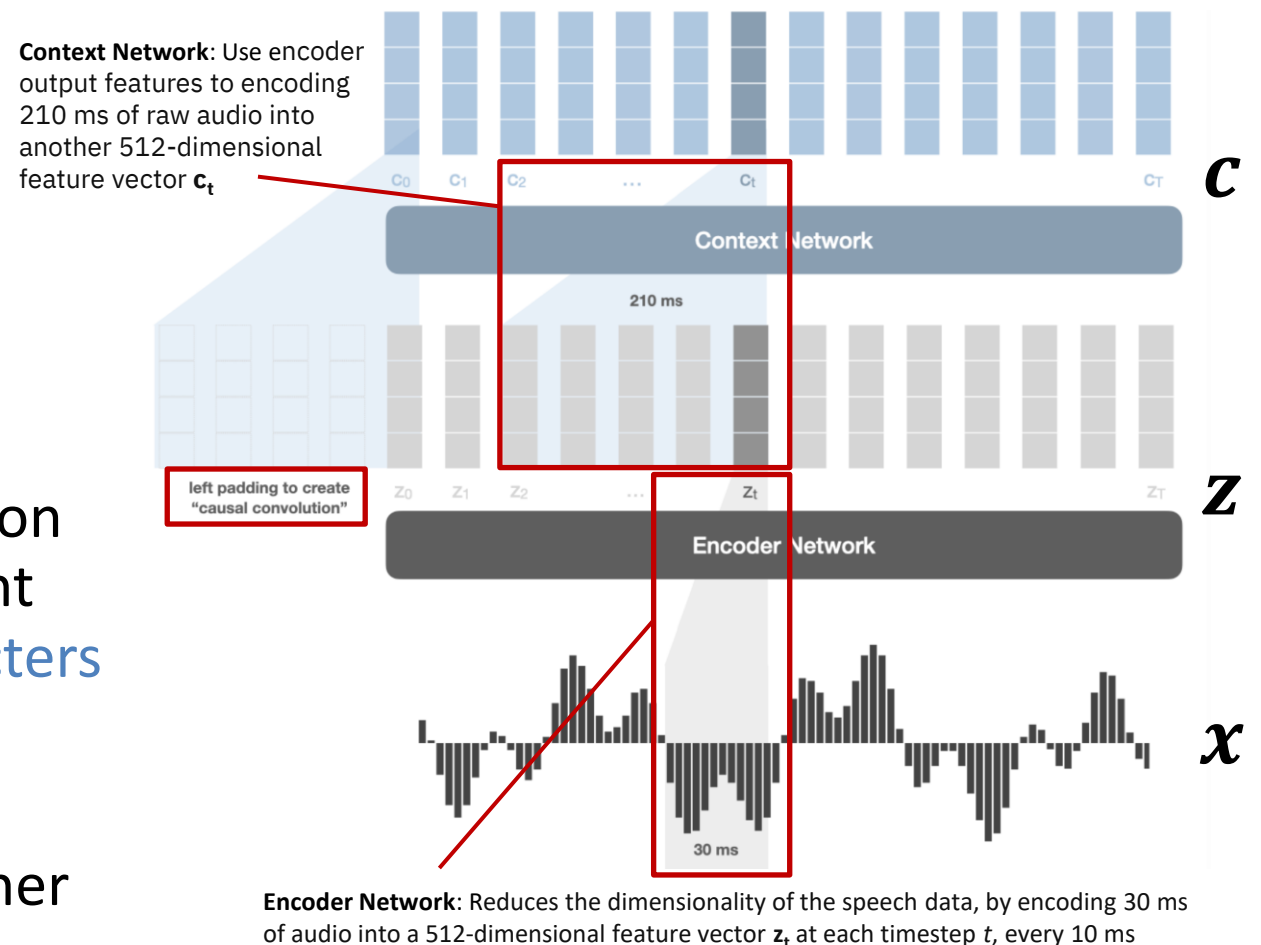


Processes log-mel signal via low-level feature encoder and transformer encoder with auto-regressive decoder

3. SELF-SUPERVISED LEARNING IN THE WAV2VEC 1.0 & 2.0 ENCODER

Automatic speech recognition with wav2vec

- Exploit easy-to-collect unlabeled audio data
- Apply self-supervised learning in the encoder with causal CNNs
 - Input: waveform audio data
 - Output: latent representation c_i
- Fine-tune system after pre-training on smaller, labeled datasets using latent representations c_i to predict characters and words...
- wav2vec outperforms RNN-based encoders and is easy to adapt to other domains / languages



Automatic speech recognition with wav2vec

- Encoder network $f: X \rightarrow Z$
 - causal 1D CNN with five-layers, 5 strided kernels, 512 channels
- Context network $g: Z \rightarrow C$
 - causal 1D CNN with nine-layers, strided kernels, 512 channels
- Objective is a contrastive loss:
 - maximize mutual information between context features and K future audio samples using ten negative samples
 - distractors sampled from same sequence & speaker; max. prediction horizon set to $K = 12$ with different transformations h_k

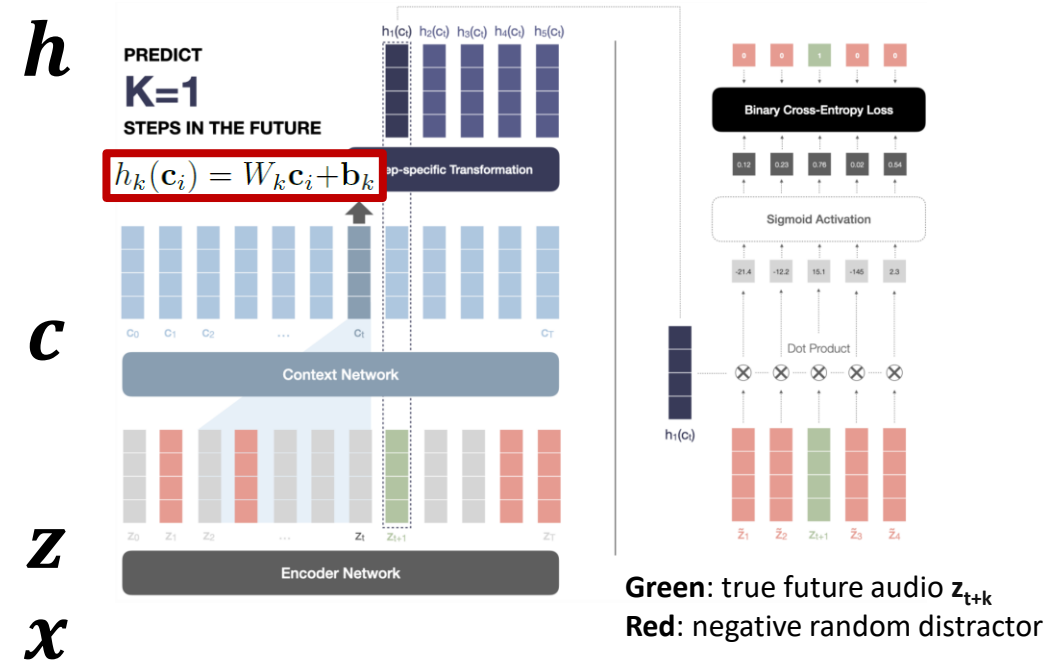


Image: J Bgn: The illustrated wav2vec

$$\mathcal{L} = \sum_{k=1}^K \mathcal{L}_k$$

$$\mathcal{L}_k = - \sum_{i=1}^{T-k} \left(\underbrace{\log \sigma(\mathbf{z}_{i+k}^\top h_k(\mathbf{c}_i))}_{\text{trained to be high}} + \lambda \mathbb{E}_{\tilde{\mathbf{z}} \sim p_n} \underbrace{\log \sigma(-\tilde{\mathbf{z}}^\top h_k(\mathbf{c}_i))}_{\text{trained to be low for distractors}} \right)$$

$$P(y = 1 \mid z, c_i) \qquad P(y = 0 \mid \tilde{z}, c_i) = 1 - \sigma(\tilde{z}^\top h_k(c_i)) = \sigma(-\tilde{z}^\top h_k(c_i)) \approx 1$$

Automatic speech recognition with wav2vec

- Encoder network $f: X \rightarrow Z$

From the paper: “The encoder layers have kernel sizes (10, 8, 4, 4, 4) and strides (5, 4, 2, 2, 2). The output of the encoder is a low frequency feature representation $z_i \in Z$ which encodes about 30 ms of 16 kHz of audio and the striding results in representations z_i every 10 ms”

- Explanation:

- At 16 kHz, 1 ms correspond to 16 samples
- Spacing between adjacent outputs is the product of strides at layer l , i.e., $j_l = s_0 \cdot s_1 \cdot s_2 \cdot \dots \cdot s_l$
- Receptive field, i.e., number of original input samples influencing one output: $r_l = r_{l-1} + (k_l - 1) \cdot j_{l-1}$
- each final encoder output z_i covers 465 samples i.e., 29.1 ms, total stride j_l equals 160 or 10 ms

Layer	Kernel	Stride	Jump in input samples	Receptive field	
1	10	5	5		10
2	8	4	$5 \cdot 4 = 20$	$10 + (8 - 1)5 = 45$	
3	4	2	40	$45 + (4 - 1)20 = 105$	
4	4	2	80	$105 + (4 - 1)40 = 225$	
5	4	2	160	$225 + (4 - 1)80 = 465$	

Automatic speech recognition with wav2vec

- Encoder network $f: X \rightarrow Z$

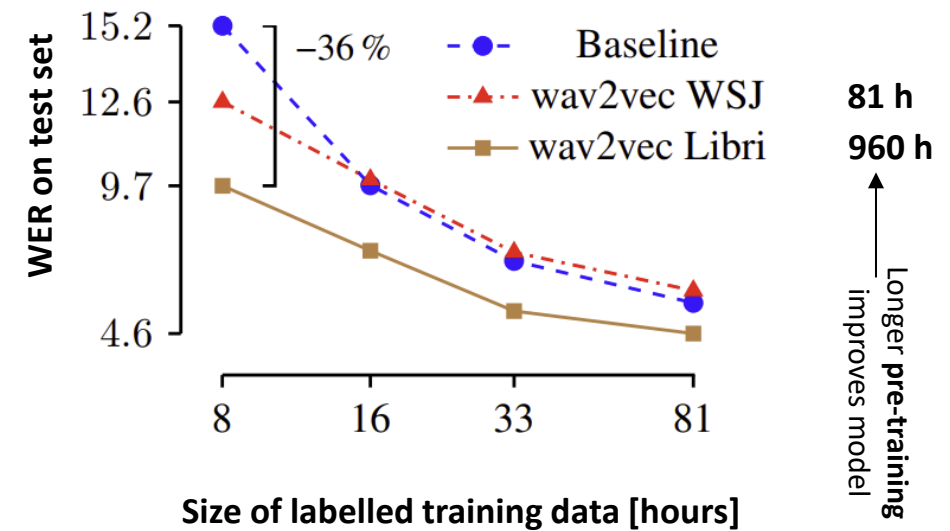
From the paper: “To train the baseline acoustic model we compute *80 log-mel filterbank coefficients for a 25 ms sliding window with stride 10 ms.*”

- Explanation:
 - At 16 kHz sampling rate a 25 ms segment contains 400 audio samples and each 25 ms segment is assumed short enough that its frequency content is approximately stable
 - For each 25 ms segment a spectrum is computed and passed through 80 mel-spaced filters, i.e., for each segment the model obtains numbers measuring energy in 80 perceptual frequency bands measured in mel units

Automatic speech recognition with wav2vec

- Unlike RNN, wav2vec's fully convolutional architecture can be easily parallelized
- Learned wav2vec representations are fed into supervised ASR system
 - i.e., the output of the context network c_i , a vector with 512 dimensions per audio frame are entered into the acoustic model
 - decoding: uses a pronunciation model, combined with a language model applying beam search
- Learned wav2vec representations from ~ 1000 hours of unlabeled speech outperform the best ASR baseline on WSJ data (WER: 2.43% vs. 3.1%)

- Impact of wav2vec pretraining compared to Baseline using log-mel filterbank features



Automatic speech recognition with wav2vec2

- Exploit easy-to-collect unlabeled audios and apply self-supervised learning with contrastive loss
- Use
 - Transformer encoder to build contextualized representations
 - Product quantization to learn finite set of speech representations
 - Input: waveform audio data
 - Output: latent representation c_i
- Fine-tune system after pre-training on smaller, labeled datasets using latent representations c_i

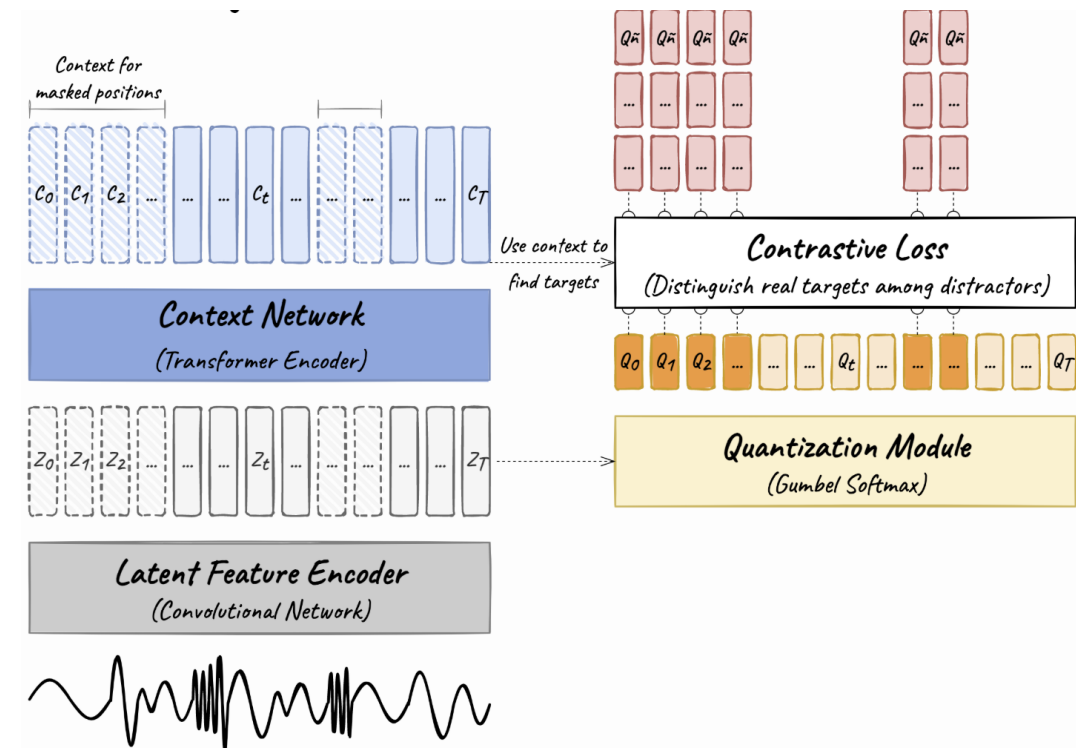
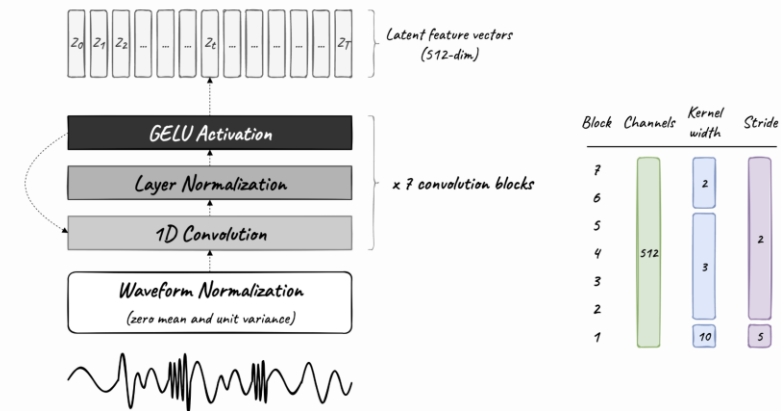


Image: J Bgn: The illustrated wav2vec2

Automatic speech recognition with wav2vec2

- Latent feature encoder network $f: X \rightarrow Z$
 - Reduce dimensionality of audio data
 - 1D CNN with seven-layers, 7 strided kernels, 512 channels
 - outputs an embedding z_i every 20 ms based on 25 ms of audio
- Context network Transformer $g: Z \rightarrow C$
 - Learn contextualized representation c_i over the entire sequence of latent representations
 - Encoder blocks (12 or 24), 768 dimensions, 8 heads, 3072 dimensions in FFNN with fixed relative positional embeddings via 1D CNN
 - Ca. 49% of all time steps masked with a mean span length of 299 ms

Wav2vec 2.0 Latent Feature Encoder



Wav2vec 2.0 Context Network (Transformer Encoder)

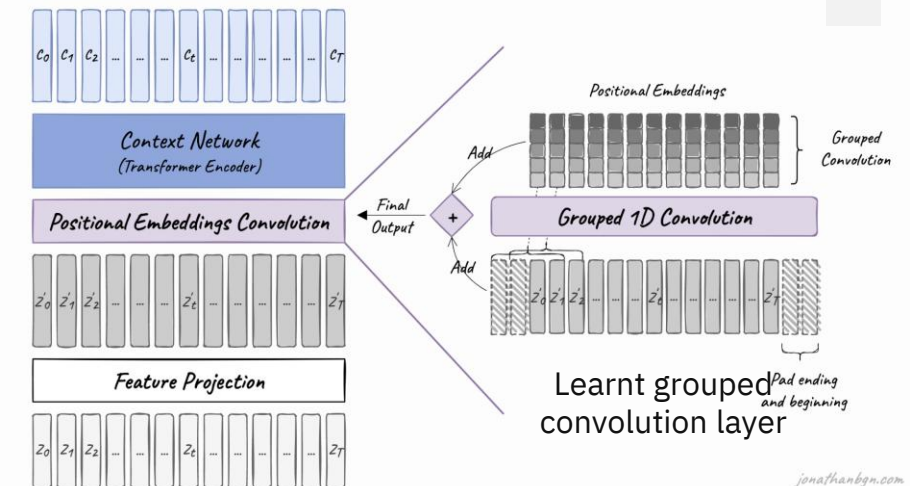


Image: J Bgn: The illustrated wav2vec2

Automatic speech recognition with wav2vec2

- Encoder network $f: X \rightarrow Z$

“The feature encoder contains seven blocks and the temporal convolutions in each block have 512 channels with strides (5,2,2,2,2,2,2) and kernel widths (10,3,3,3,3,2,2). This results in an encoder output frequency of 49 Hz with a stride of about 20 ms between each sample, and a receptive field of 400 input samples or 25 ms of audio.”

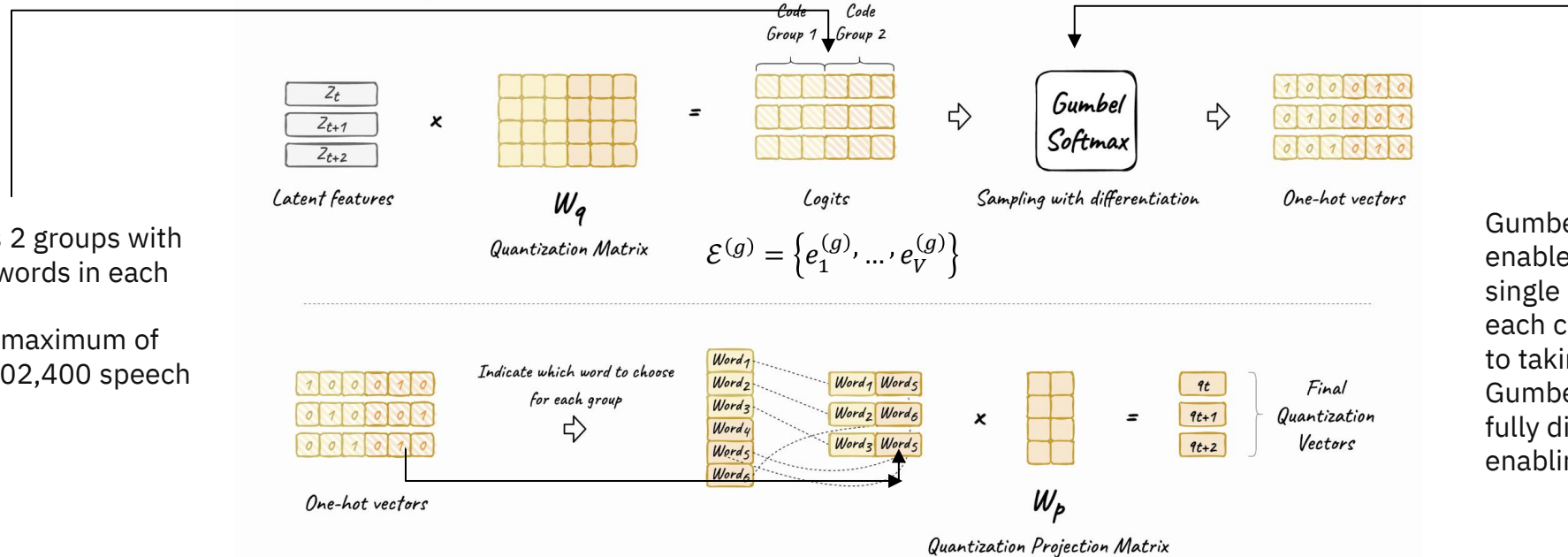
- Explanation:
 - The total stride of stacked convolutions is $5 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 \cdot 2 = 320$, i.e., each encoder output step advances by 320 input samples. At 16 kHz, 320 samples account for $320/16000 \text{ s} = 20 \text{ ms}$ between successive decoder outputs or a frequency of 50 Hz
 - After stacking several layers, and because convolution output length uses integer flooring, i.e. the exact output count for 1 second of audio (16000 samples) becomes slightly less than 50.

For a 1D convolution $L_{out} = \left\lfloor \frac{L_{in} - k}{s} \right\rfloor + 1$

Automatic speech recognition with wav2vec2

- Quantization module $q: Z \rightarrow Q$
 - Learn discrete speech units from unlabelled data, i.e. discretize z_i into q_i with product quantization and remove tiny acoustic variations, irrelevant detail, noise, exact waveform

Wav2vec 2.0 Quantization Module



Wav2vec uses 2 groups with 320 possible words in each group,
→ theoretical maximum of $320 \times 320 = 102,400$ speech units

Gumbel softmax enables sampling a single codeword from each codebook similar to taking argmax but Gumbel softmax is fully differentiable enabling backprop

Automatic speech recognition with wav2vec2

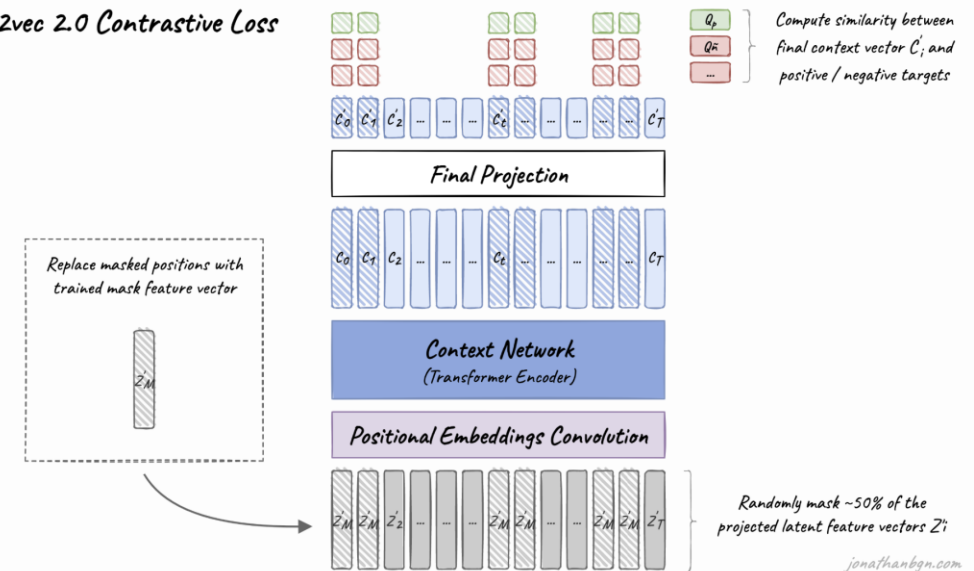
- Objective function / contrastive loss

- $\mathcal{L}_m$: Identify correct quantized representation q_t among a set of $K + 1$ quantized latent speech representations $\tilde{q}$ including K distractors for a given context network output c_t centered over masked time step t ($K = 100$, $\kappa = 0.1$, $\text{sim} = \text{cosine}$)
- $\mathcal{L}_d$: add codebook diversity loss to encourage the model to use the PQ codebook entries equally often

$$\mathcal{L} = \mathcal{L}_m + \alpha \mathcal{L}_d$$

$$\mathcal{L}_m = -\log \frac{\exp(\text{sim}(\mathbf{c}_t, \mathbf{q}_t)/\kappa)}{\sum_{\tilde{\mathbf{q}} \sim \mathbf{Q}_t} \exp(\text{sim}(\mathbf{c}_t, \tilde{\mathbf{q}})/\kappa)}$$

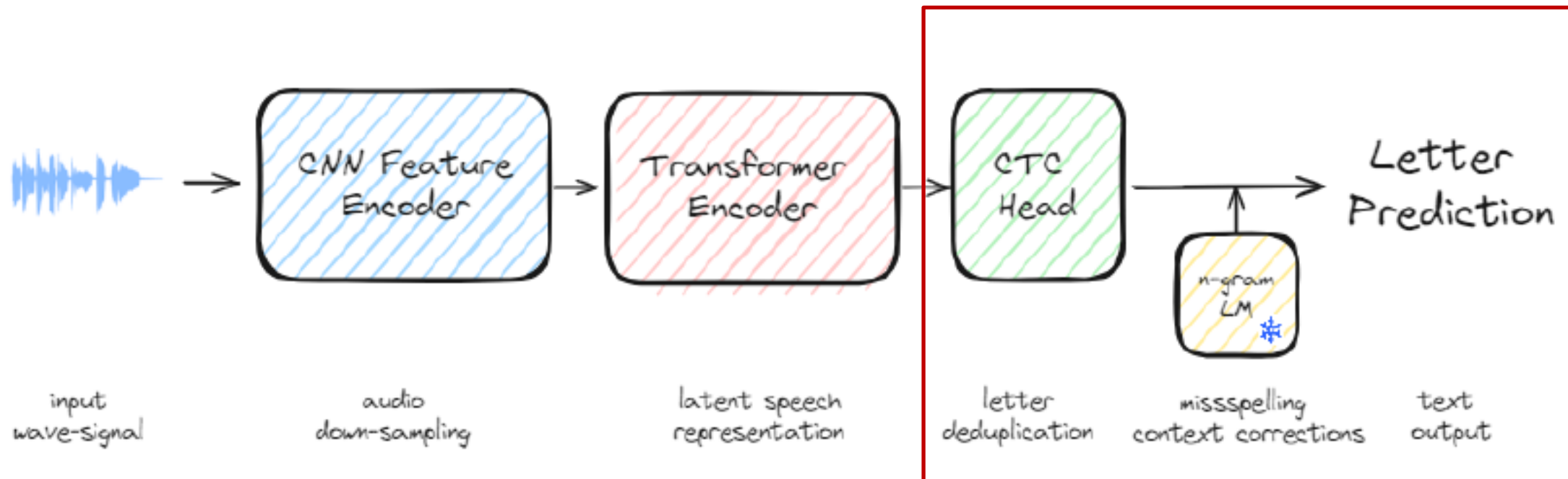
Wav2vec 2.0 Contrastive Loss



Results of wav2vec2

- Wav2vec2 jointly learns contextualized speech representations c_t and an inventory of discretized speech units q_t
- Learned wav2vec2 representations are fed into supervised ASR system, i.e. fine-tuned on labeled data (audio + transcript)
 - linear layer added on top of the Transformer with 30 outputs = 26 letters plus ' , . sep
 - Model optimized by minimizing a CTC loss combined with probabilities from a language model
- Learned wav2vec2 representations, followed by fine-tuning,
 - outperform SOTA on full Librispeech benchmark (noisy speech WER: 3.3%, clean speech WER: 1.8%) while using 100 times less labeled data
 - yields excellent WER for Librispeech (clean : 4.8%; noisy 8.2%) with only 10 minutes of transcribed speech

Inference with wav2vec2 (simplified)



Why an LM? ...Because of introduction of wrong words

"... at this festive season of the year with **christmaus** and..." → inexistent: **christmaus**

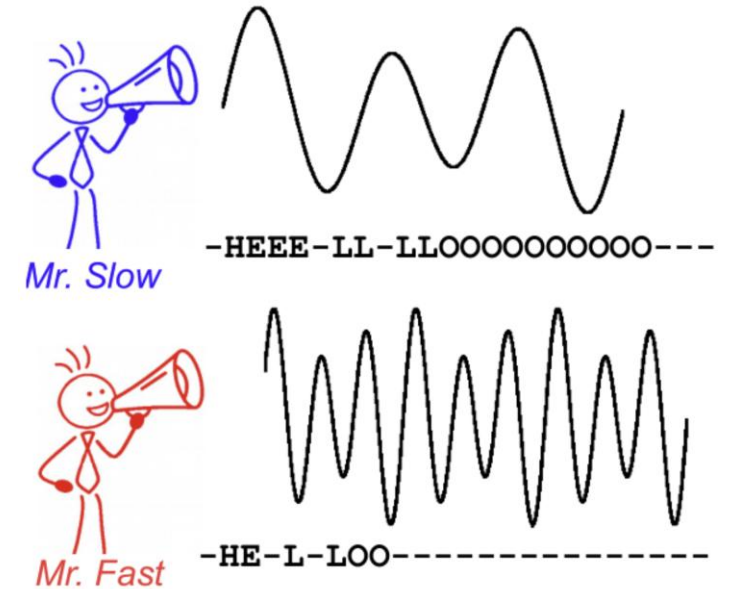
"... and **rose beef** looming before us ..." → out-of-context: **rose** instead of **roast**

Why a n-gram LM?... Because Transformer-based LMs are computationally very expensive

4. CTC LOSS AND CTC DECODING

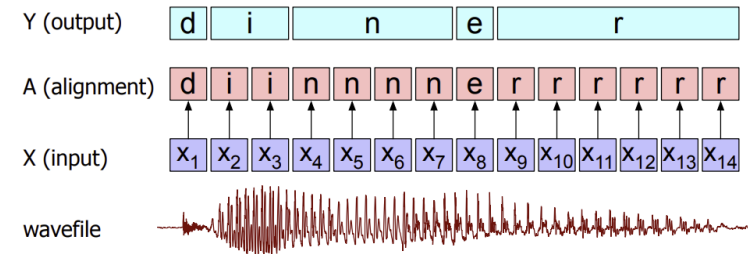
Motivation for CTC

- ASR peculiarities
 - **Imbalance:** long acoustic input sequence X mapping to a much shorter sequence of letters Y
 - **Misalignment:** mapping parts of X to parts of Y is ambiguous
- If a system recognizes a letter for every frame, we need a merging strategy to obtain words, because several mergers may lead to the same words
- Intuition for [Connectionist Temporal Classification](#)
 - output a single character for every frame of the input
→ alignment A has the same length as input X
 - apply a collapsing function that combines sequences of identical letters, resulting in a shorter sequence Y

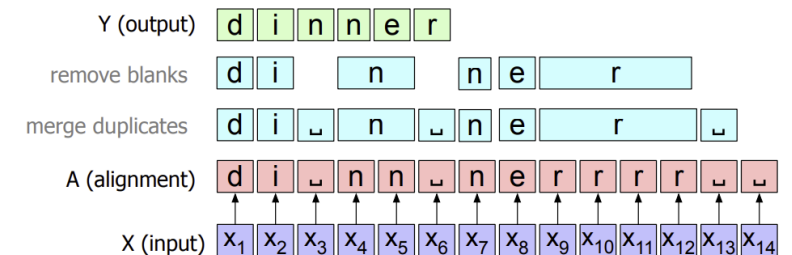


CTC collapsing function

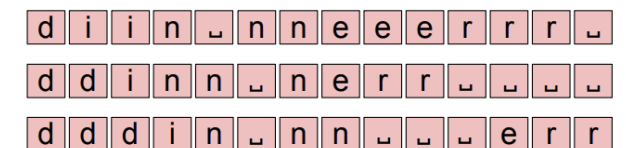
- Main hypothesis: for each audio frame, an encoder outputs a probability distribution over the possible characters (letters, word separator, punctuation)
- CTC
 - Adds a **symbol** called ‘blank’ (noted as ‘_’) to the transcription alphabet as **additional predict token**
 - Defines **collapsing function** $B: a \rightarrow y$ between alignment and output which
 - collapses all repeated symbols
 - then removes all blanks symbols
- The **CTC collapsing function is many-to-one**; lots of different alignments map to the same output string.



Naive



CTC



m:1

CTC training loss

- For neural network encoders, the collapsing function is used to define a **loss**, based on probabilities assigned to the correct sequences of letters, over the train set D
- $$L_{CTC} = \sum_{(X,Y) \in D} -\log P_{CTC}(Y|X) = \sum_{(X,Y) \in D} -\log \sum_{all\ valid\ alignments} \prod_{t=1}^T p(a_t | h_t)$$
 - 1st equation: sum of the negative log-likelihoods of the correct output Y for each input X
 - 2nd equation: most probable output Y determined via highest sum over the probability of all possible alignments resulting in Y
 - 2nd equation: CTC output a_t at time t is independent of output labels at any other time a_i ($i \neq t$) & dependent on encoder state h_t
- Note: **CTC loss** is **differentiable** but can be very expensive to **compute** → **dynamic programming** with a forward-backward algorithm (see CTC paper)

CTC inference decoding

- After training, we use the model to infer a likely Y given an X by maximizing the conditional probability $p(Y|X)$
- CTC works by summing over the probability of all possible alignments between Y and X
- CTC does not implicitly learn a language model over the data due to conditional independence assumption of alignments a_t across time t , hence modify CTC score and include language model and length term with λ_i optimized on validation set

Example (alignments for one time-step)

Given $p([b, b, b]) > p([a, a, _])$ and

$p([b, b, b]) > p([a, a, a])$ and

$p([b, b, b]) < p([a, a, _]) + p([a, a, a])$

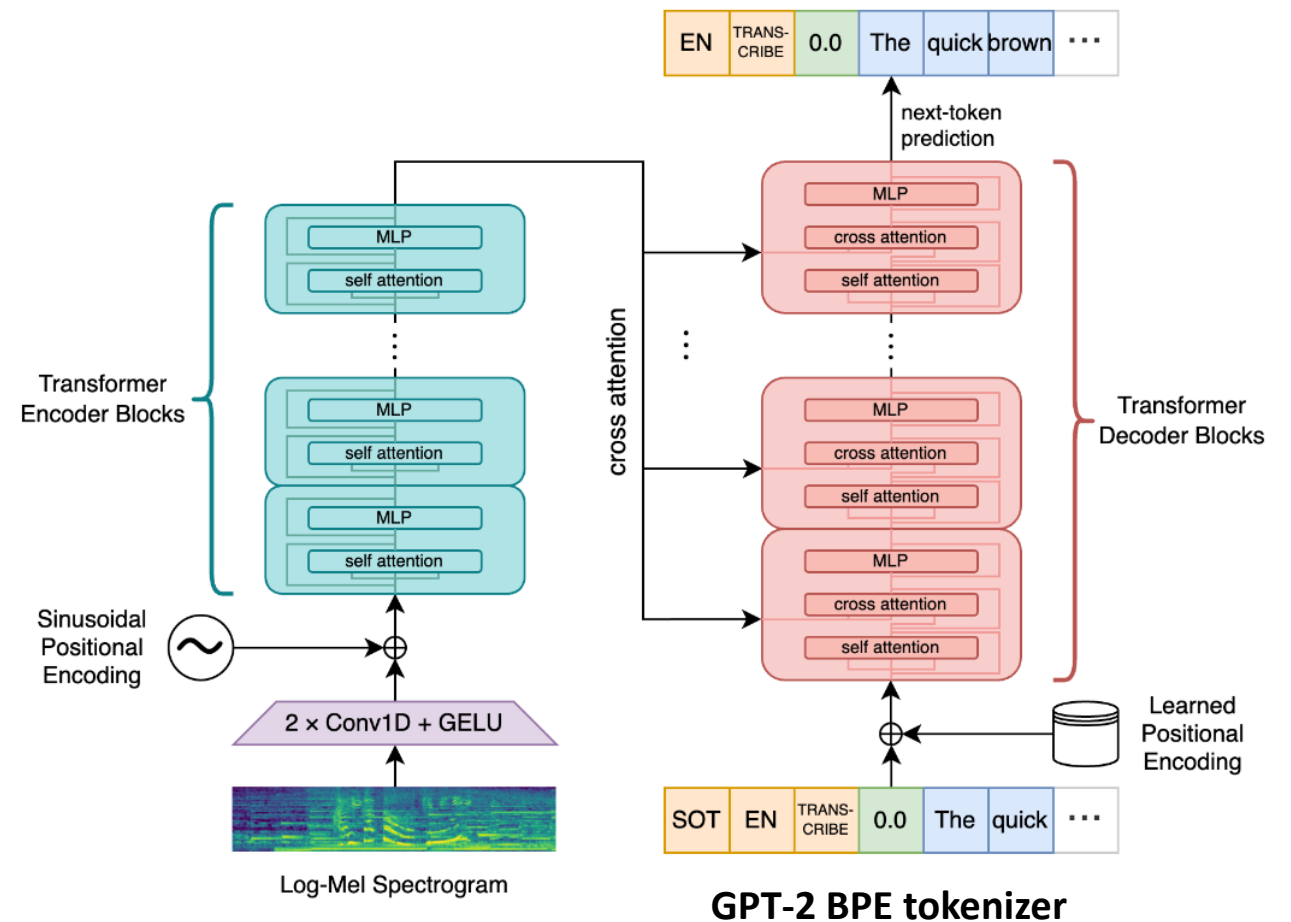
→ Choose $Y = [a]$ and not $Y = [b]$

$$score_{CTC} = \log P_{CTC}(Y|X) + \lambda_1 \cdot \log P_{LM}(Y) + \lambda_2 \cdot L(Y)$$

5. ENCODER-DECODER MODEL: WHISPER

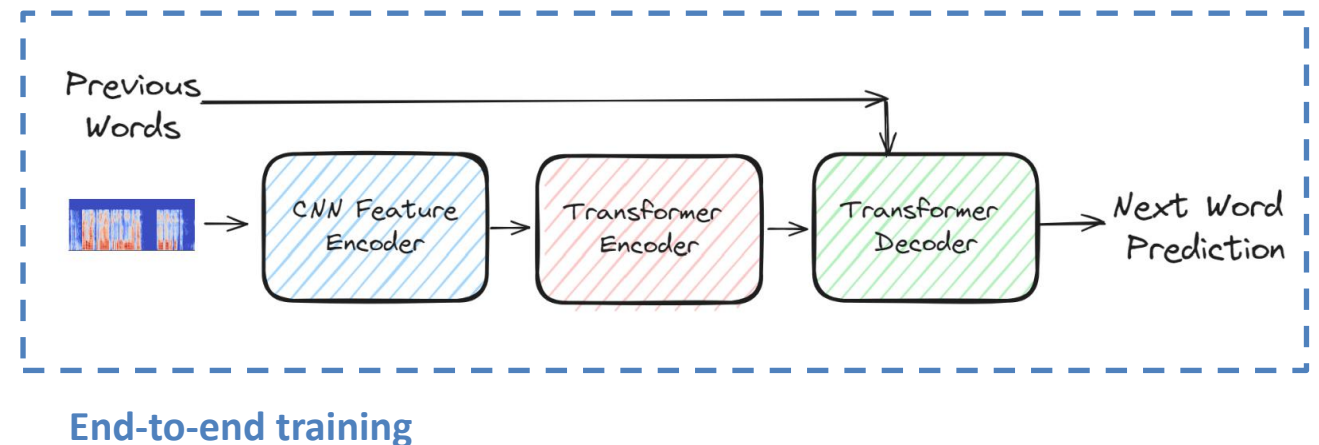
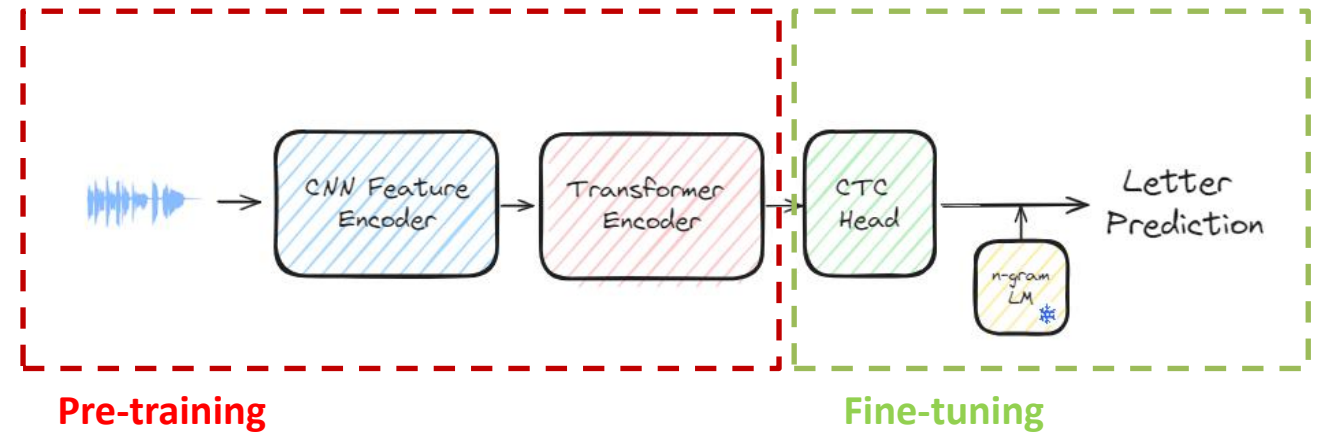
Automatic speech recognition with Whisper

- Exploit full **end-to-end training of encoder-decoder transformer**
 - Input: log-mel spectrogram audio features (80 channels, 25 ms frames)
 - Output: autoregressive decoder with GPT-2 BPE vocabulary tokens
- Collect **massive amounts of weakly supervised labeled audio data**
- Use **multilingual, multitask learning to develop a robust ASR system** applicable in a broad range of environments



Whisper's training paradigm

- wav2vec 2.0:
 - unsupervised pre-training clearly improved audio encoders quality
 - Lack of high-quality pre-trained decoder and recommended protocol of dataset-specific finetuning
- Whisper:
 - ASR systems pre-trained in a supervised fashion across many datasets/domains exhibit higher robustness



Whisper's large-scale weak supervision data sets

- Massive **web-scraping** with a **minimalist approach** to data pre-processing
 - Collect **audio-transcript pairs** of **high diversity** (speakers, 99 languages, recording quality / noise levels)
 - Remove output of existing **ASR models** by applying heuristics to transcripts (all-uppercase, all-lowercase, no commas or complex punctuation)

English transcription

438'000 hours
(65%)

Non-English transcription

117'000 hours
(17%)

Any-to-English translation

126'000 hours
(18%)

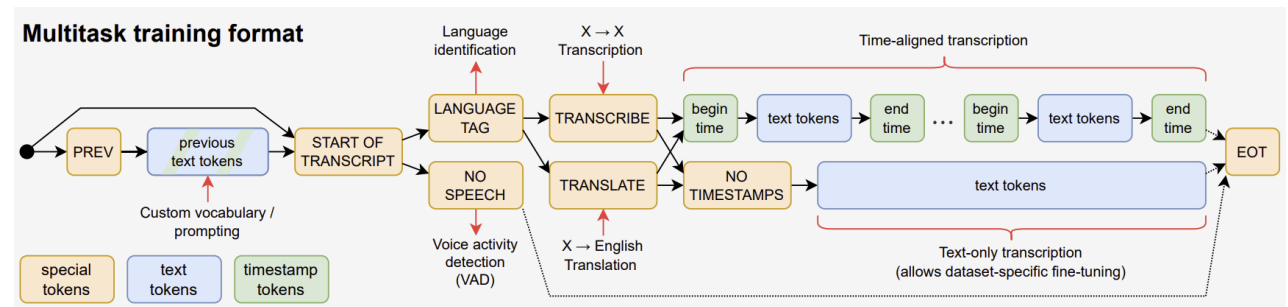
No speech identification

Identified via voice
activity detection

Whisper's multitask learning

- Based on this data, Whisper learns several tasks **in the decoder**
 - Language identification
 - Voice activity detection (voice Y/N)
 - Transcription (Audio $\rightarrow$ Text)
 - Translation (Audio $\rightarrow$ EN)
 - Alignment/Time stamp prediction (time resolution 20 ms)

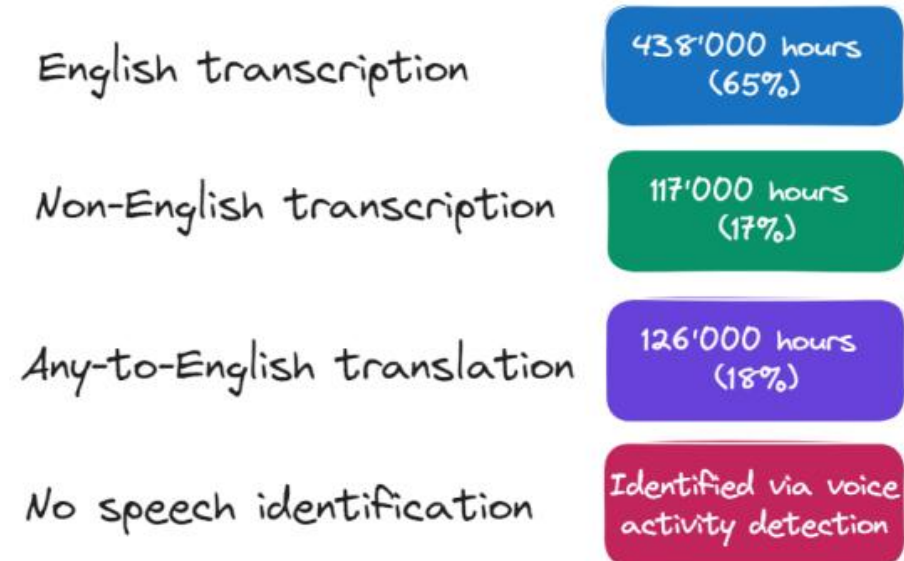
Special tokens control training and fine-tuning of the decoder, e.g., language tag “DE”, no_timestamp_training = False



- Learning ambiguous audios is enabled by conditioning on the history of text transcripts

Whisper fine-tuning for CH-German

Whisper



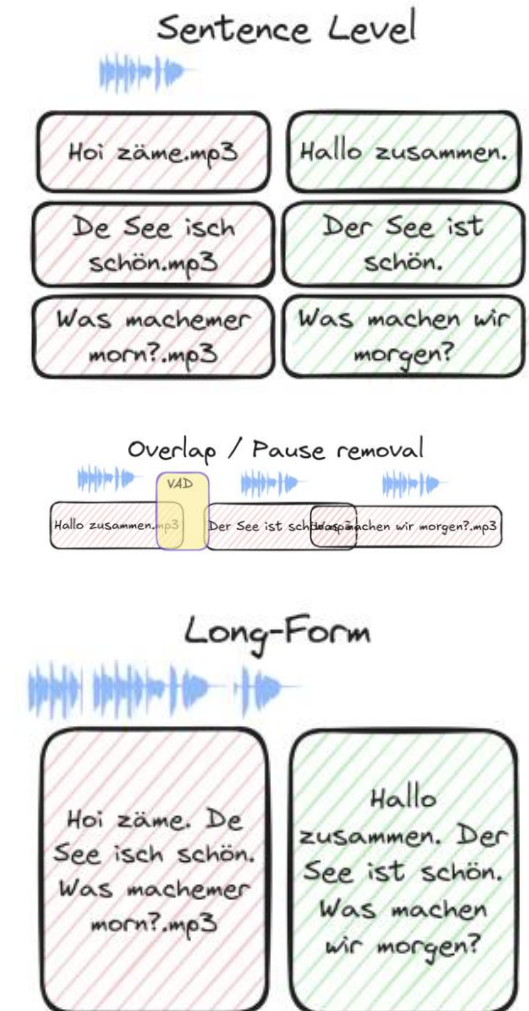
CH-German



- Whisper was trained on 30-second audio segments paired with the corresponding transcript... available CH-German audio-text data has much shorter segments

Whisper fine-tuning for CH-German

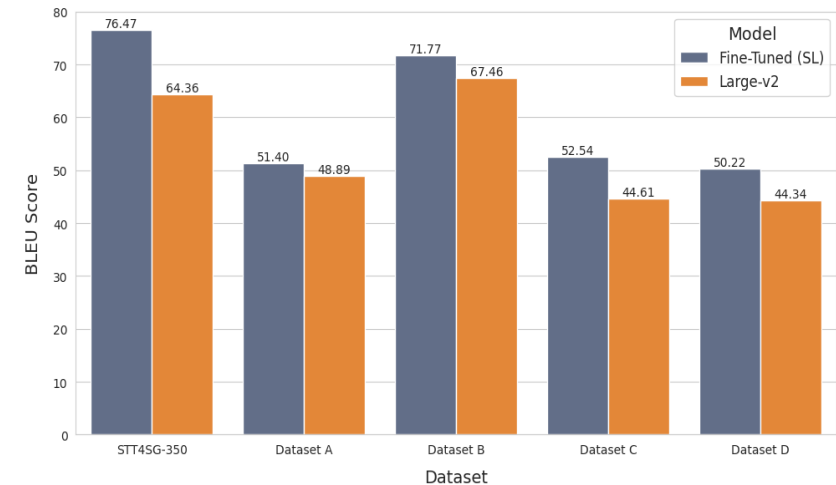
- Fine-tuning is specified as “Transcription” task, although we translate Swiss-German audio to German text
- Fine-tuning on sentence-level data loses time-stamp information and does not generalize to long text
- Key idea: generate synthetic Long-Form Data with
 - Timestamp correction using Voice Activity Detection (VAD)
 - Noise overlapping and negative overlap to simulate realistic audio transitions
 - Speaker retention logic to simulate natural conversation dynamics



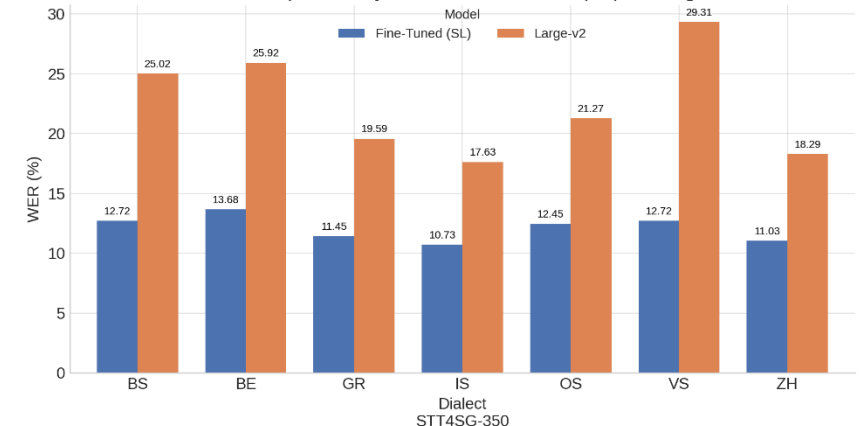
Whisper fine-tuning for CH-German

- Our fine-tuned Whisper surpasses Whisper Large-v2
 - on all longer Swiss-German datasets evaluated
 - with respect to Swiss named-entities (e.g., “Brugg” transcribed as “Brugg” not “Brücke”!)
 - consistently on all main Swiss-German dialects for which data is available (in particular for “Oberwalliser-Tiitsch” and dialects from Bern and Basel)

BLEU Score Comparison: Fine-Tuned (SL) vs Large-v2



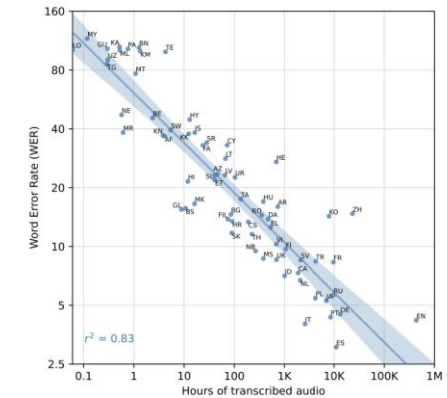
WER Comparison by Dialect: Fine-Tuned (SL) vs Large-v2



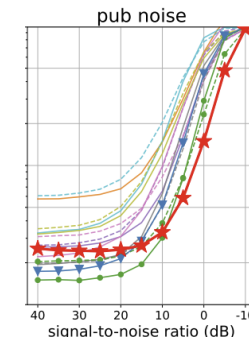
Overall assessment of Whisper

- Whisper represents a robust out-of-the box solution
 - for transcription of English & many well-resourced languages (top-10 languages WER < 7%)
 - for translation of audios into English
 - WER varies a lot (bottom-10 languages score at only 40%-55%)
 - resilient to various accents, technical jargon and background noise (see comparison to wav2vec2, conformer, huBERT)
 - with open problems (e.g., repeat loops, not transcribing the first or last few words of an audio) ...hallucinating text completely unrelated to audio input

WER vs training audio hours
(for Transcription)



WER vs SNR under pub noise
(LibriSpeech test-clean)



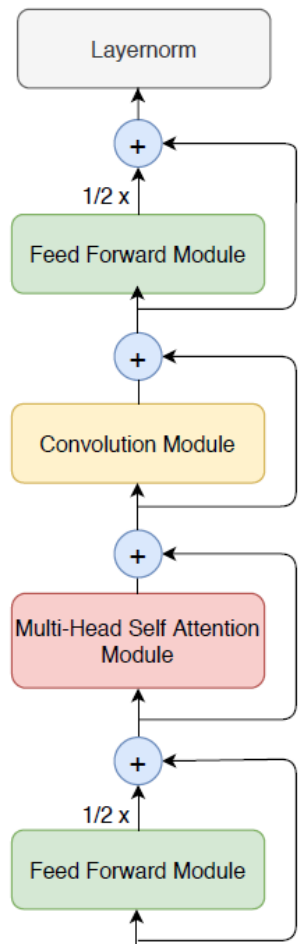
Whisper ★

Whisper models and scores

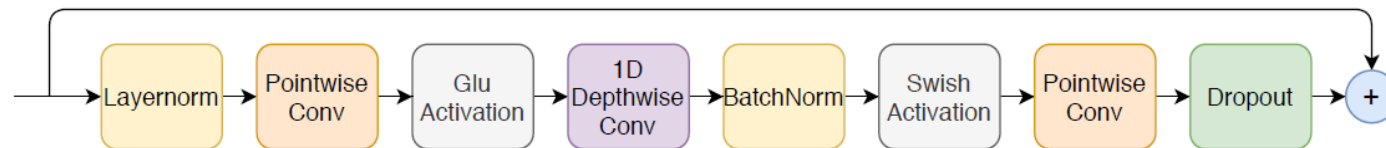
- Released open weight models on Hugging Face (Apache 2.0 license, ok for commercial use) with [code](#) on GitHub
 - Whisper Large V2 (December 2022)
 - [Whisper Large V3](#) (November 2023)
- [Available models and languages](#)

Size	Parameters	English-only model	Multilingual model	Required VRAM	Relative speed
tiny	39 M	<code>tiny.en</code>	<code>tiny</code>	~1 GB	~10x
base	74 M	<code>base.en</code>	<code>base</code>	~1 GB	~7x
small	244 M	<code>small.en</code>	<code>small</code>	~2 GB	~4x
medium	769 M	<code>medium.en</code>	<code>medium</code>	~5 GB	~2x
large	1550 M	N/A	<code>large</code>	~10 GB	1x
turbo	809 M	N/A	<code>turbo</code>	~6 GB	~8x

Conformer – Transformer block with convolution



- Often trained from scratch (CTC loss or cross-entropy), good accuracy and latency, used in the industry
 - Whisper: useful as out-of-the box, thanks to pre-training
- Several implementations available: e.g. in [ESPnet](#) or [NeMo](#)
- Differences from Whisper functionalities
 - not natively multilingual, no translation
 - no language detection, no timestamps



Source: Gulati et al., [Conformer: Convolution-augmented Transformer for Speech Recognition](#), Interspeech, 2020.

6. EVALUATION AND CURRENT PERFORMANCE LEVELS

Evaluation of ASR

- Word Error Rate (WER)
 - counts substitutions, deletions and insertions with respect to a reference transcript
 - Ranges from 0 (perfect transcript) to over 100%
- WER only measures transcription accuracy
 - all errors are equally weighted
 - semantic distance is not considered
 - can be made more robust by normalizing ASR output and reference (digits, fillers, contractions)
 - ...
 - Character Error Rate (CER) is more tolerant for small mistakes (phoneme-level errors in words)

Reference	Candidate
The bard sang ancient melodies of nature, transforming tranquil meadows into sonnets for enhanced soulful grace.	The poetic bard echoed ancient melodies, transcending meadows into sonnets for enhanced soulful grace.

Step 1. Count Errors

Highlighting the **substitution**, **deletion**, and **insertion** errors, we can count each type of error:

The **poetic** bard **echoed** ancient melodies **of nature**, **transcending** **tranquil** meadows into sonnets for enhanced soulful grace.

In our candidate text, we have 2 substitutions, 3 deletions, and 1 insertion.

Step 2. Calculate WER

With each errors counted, we can calculate our WER. Using the formula,

$$\begin{aligned}
 \text{WER} &= \frac{\text{Substitutions} + \text{Deletions} + \text{Insertions}}{\text{\# of Words in Reference}} \\
 &= \frac{2 + 3 + 1}{16} \\
 &= \frac{6}{16} \\
 &= 0.375
 \end{aligned}$$

we arrive at a WER of 0.375 for our candidate text.

Evaluation of ASR

Reference	Candidate
The bard sang ancient melodies of nature, transforming tranquil meadows into sonnets for enhanced soulful grace.	The poetic bard echoed ancient melodies, transcending meadows into sonnets for enhanced soulful grace.

Step 1: Tokenization & n-grams

Splitting our sentences up into 1-, ..., 4-grams, we get:

Step 2: Calculate n-gram Overlap

Next, let's calculate the clipped precision scores for each of the n-grams. Recall that the precision formula is:

$$p_i = \frac{\text{Clipped count of matching i-grams in machine-generated text}^1}{\text{Total number of i-grams in machine-generated text}}$$

Step 3: Calculate Brevity Penalty

We apply a brevity penalty to prevent the BLEU score from giving undeservingly high scores for short generated texts. Recall that our formula is:

$$\min \left(1, \exp \left(1 - \frac{\text{reference length}}{\text{output length}} \right) \right) = \min \left(1, \exp \left(1 - \frac{9}{8} \right) \right) = 0.882$$

Step 4: Calculate BLEU

Combining our n-gram overlap and Brevity Penalty, our final BLEU score is:

$$\text{BLEU} = \text{Brevity Penalty} \times \text{n-gram Overlap} = 0.882 \times 0.841 = 0.742$$

Reference	Candidate
The bard sang ancient melodies of nature, transforming tranquil meadows into sonnets for enhanced soulful grace.	The poetic bard echoed ancient melodies, transcending meadows into sonnets for enhanced soulful grace.

Step 1. Count Errors

Highlighting the **substitution**, **deletion**, and **insertion** errors, we can count each type of error:

The poetic bard echoed ancient melodies of nature, transcending tranquil meadows into sonnets for enhanced soulful grace.

In our candidate text, we have 2 substitutions, 3 deletions, and 1 insertion.

Step 2. Calculate WER

With each errors counted, we can calculate our WER. Using the formula,

$$\begin{aligned} \text{WER} &= \frac{\text{Substitutions} + \text{Deletions} + \text{Insertions}}{\# \text{ of Words in Reference}} \\ &= \frac{2 + 3 + 1}{16} \\ &= \frac{6}{16} \\ &= 0.375 \end{aligned}$$

we arrive at a WER of 0.375 for our candidate text.

Source: [Kolena - WER, CER, MER](#)

Examples of speech data

- **LibriSpeech**: large open-source read-speech dataset (16 kHz, 1000 hours), audio books from the LibriVox project, with transcripts aligned at the sentence level (Panayotov et al., 2015)
 - divided into an easier (“clean”) and a more difficult portion (“other”) for evaluation
 - clean portion = higher recording quality & accents closer to US English
- **Mozilla Common Voice**: publicly available dataset from volunteers, > 100 languages
 - English: 93k speakers, 3.5k hours, 1.67M sentences; French: 19k speakers, 1.15k hours, 1.64M sentences
- **TIMIT**: recordings of 630 speakers in 8 variants of American English (1986), each reading ten phonetically rich sentences
- **Switchboard**: prompted telephone conversations between strangers, with 2430 conversations averaging 6 minutes each (8 kHz, 240 hours, 3M words) (Godfrey et al., 1992)
 - many auxiliary handmade annotations: parse trees, dialogue acts, phonetic and prosodic labels
- **CALLHOME**: 120 unscripted 30-minute telephone conversations between native speakers of English, usually close friends or family (Canavan et al., 1997)

Best scores of recent ASR systems

Model	Architecture	Extra Training Data	WER% ↓	WER% ↓	Model	Architecture	Extra Training Data	WER% ↓
LibriSpeech test					TIMIT			
			clean	others				
Conformer + Wav2vec 2.0 [658]	Conformer + wav2vec2.0	Y	1.4	2.6	wav2vec 2.0 [26]	Transformer + CNN	Y	8.3
w2v-BERT XXL [96]	CNN+Transformer	Y	1.4	2.5	vq-wav2vec [25]	Transformer + CNN	Y	11.6
SpeechStew (1B)[58]	Conformer	Y	1.7	3.3	LSTM + Monophone Reg [455]	LSTM	N	14.5
SpeechStew (100M) [58]	Conformer	N	2.0	4.0	Common Voice			
ContextNet + SpecAugment [413]	LSTM+CNN	Y	1.7	3.4	SpeechStew (1B) [58]	Conformer	N	10.8
Conformer (L) [162]	Conformer	N	1.9	4.1	Whisper [443]		N	9.5
ContextNet [169]	Conformer + wav2vec2.0	N	1.9	3.4	WSJ eval92			
Squeezeformer [255]	Conformer	N	2.47	5.97	SpeechStew (100M) [58]	Conformer	N	1.3
LSTM Transducer [636]	LSTM	N	2.23	5.6	tdnn+chain [433]	TDNN	N	2.32
Transformer Transducer [331]	Transformer	N	2.0	4.2	GigaSpeech			
Whisper [443]		N	2.7 (ZS)	5.6 (ZS)	Conformer/Transformer-AED [61]	Conformer	N	10.80

Source : A. Mehrish et al., [A Review of Deep Learning Techniques for Speech Processing](#), 2023